P2 DAVID RICO ALG

1)INTERCAMBIO DIRECTO o BURBUJA N = 1

n	T ordenado	T inverso	T aleatorio
1000	455	1090	886
2000	1724	4394	3447
4000	6747	17567	13821
8000	27317	69867	55271
16000	107874	278160	
32000	FDT	FDT	FDT
64000	FDT	FDT	FDT
128000	FDT	FDT	FDT

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada:

Si que concuerdan ya que los datos reflejan un comportamiento cuadrático, O(n^2). Con respecto a las diferencias de ordenación, el ordenado es más rápido ya que evalúa la condición if (a[j - 1] > a[j], pero nunca entra dentro, sin embargo en las otras ordenaciones si que entra y tarda más.

2)SELECCIÓN N = 1

n	T ordenado	T inverso	T aleatorio
1000	394	349	389
2000	1578	1401	1570
4000	6458	5616	6331
8000	25641	22053	25197
16000	102444	88122	100381
32000	FDT	FDT	FDT
64000	FDT	FDT	FDT
128000	FDT	FDT	FDT

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada:

Si que concuerdan ya que los datos reflejan un comportamiento cuadrático, O(n^2). Con respecto a las diferencias de ordenación, en este caso el número de comparaciones dentro del algoritmo es constante y no tiene ninguna relación con el orden del array, por tanto el resultado tiene sentido.

3)INSERCIÓN N = 1

n	T ordenado	T inverso	T aleatorio
1000	O	703	342
2000	0	2737	1340
4000	1	10966	5387

8000	1	43784	21581
16000	3	172767	86386
32000	6	FDT	FDT
64000	12	FDT	FDT
128000	25	FDT	FDT

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada:

En este algoritmo podemos ver que los valores en el array ordenado es muy rápido en comparación con los demás ya que es su mejor caso y su coste es lineal O(n), en cambio su caso medio y peor es cuadrático O(n^2) y se refleja en el resultado.

4)RAPIDO N = 1

n	T ordenado	T inverso	T aleatorio
1000	3	3	4
2000	5	6	8
4000	11	11	15
8000	23	26	35
16000	49	53	71
32000	99	110	152
64000	211	230	322
128000	440	485	705

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada:

Si que concuerdan ya que los datos reflejan un comportamiento cuasi-lineal, O(n*log n). Con respecto a las diferencias de ordenación, en este caso se debe a la elección del pivote, ya que en el algoritmo cogemos el elemento central y esto facilita tanto al inverso como al ordenado, mientras que en el aleatorio tarda más.

Proyecte, a partir de las complejidades y los datos de las tablas anteriores, ¿cuántos días (1 día= 861400.000 milisegundos) tardaría cada uno de los cuatro métodos (Burbuja, Selección, Inserción y Rápido) en ordenar un vector de n= 811920.000 (o sea, n=80000*24*26=80000*210), en el caso inicialmente en orden aleatorio?

Burbuja:

T = 55.271 ms *104.857.600 = 5.795.584.409.600 ms (aprox)

Días: 5.795.584.409.600 / 86.400.000 = 67.078,5 dias (apox 183 años)

Selección:

T = 25.197 ms * 104.857.600 = 2.642.096.947.200 ms (aprox).

Días: 2.642.096.947.200 / 86.400.000 = 30.579,8 dias (aprox. 83 años).

Inserción:

T = 21.581 ms * 104.857.600 = 2.262.931.865.600 ms (aprox).

Días: 2.262.931.865.600 / 86.400.000 = 26.191,3 dias (aprox. 71 años).

Rápido:

n=128.000 es K = 81.920.000 / 128.000 = 640.

T(n_2) = T(n_1) * (n1/n2) * (log_2(n2) / log_2(n1));

log_2(128.000) = 16,96 aprox

log_2(81.920.000) = 26,28 aprox

T = 705 ms * 640 * (26,28 / 16,96) = 699.114 ms.

Días: 699.114 / 86.400.000 = 0,008 aprox (**11,6 minutos**)

5)RÁPIDO vs. INSERCIÓN para tamaños bajos para n = 100000

n	T rapido	T insercion	T rápido/T insercion
10	122	61	2
15	189	110	1,71818182
20	285	183	1,55737705
25	361	246	1,46747967
30	419	364	1,1510989
35	562	492	1,14227642
40	620	644	0,96273292
45	736	777	0,94723295
50	815	939	0,86794462
55	925	1118	0,8273703
60	1046	1319	0,79302502
65	1104	1604	0,6882793
70	1230	1844	0,6670282
75	1367	2116	0,64603025
80	1472	2342	0,62852263
85	1555	2614	0,59487376
90	1751	2943	0,59497112
95	1830	3261	0,56117755
100	1906	3637	0,52405829

Razone a partir de qué valor de n comienza a tardar menos tiempo el algoritmo de menor complejidad (rápido) que el de mayor complejidad (inserción).

En la iteración 40 el algoritmo rápido ya es más veloz que el de inserción porque cada iteración el algoritmo rápido hace una partición y cada vez tiene que recorrer la mitad de números por lo tanto es más rápido